

ControlOverWeb: İnternet Üzerinden Düşük Gecikmeli Robot Kontrolü için Çok Katmanlı Bir Sistem Mimarisi

Özet (Abstract)

Bu rapor, geleneksel robot kontrol sistemlerinin (Bluetooth, kapalı devre) menzil ve erişilebilirlik kısıtlamalarına çözüm olarak geliştirilen **ControlOverWeb** adlı bulut tabanlı robotik kontrol sisteminin teknik mimarisini detaylandırmaktadır. Projenin temel amacı, herhangi bir uygulama kurulumu gerektirmeden, standart bir web tarayıcısı üzerinden dünyanın herhangi bir yerindeki IoT cihazlarının ve robotların düşük gecikmeyle kontrol edilmesini sağlamaktır. C# .NET Core tabanlı bir backend, PostgreSQL veritabanı ve Python ile yönetilen Raspberry Pi donanım katmanından oluşan çok katmanlı bir mimari üzerine inşa edilen sistem, evrensel erişilebilirlik ve gerçek zamanlı kontrol yetenekleri sunar. Projenin en önemli teknik katkıları arasında, ağ bariyerlerini (NAT/Firewall) aşmak için kullanılan komut sorgulama (polling) mekanizması ve çoklu operatör erişiminde kontrol çakışmalarını önleyen özgün "Spy Mode" mantığı bulunmaktadır.

1. Giriş (Introduction)

Modern robotik ve IoT sistemleri alanındaki stratejik zorluk, kontrol sistemlerinin fiziksel yakınlık veya VPN gibi karmaşık ağ yapılandırmalarıyla sınırlı kalmadan evrensel olarak erişilebilir ve düşük gecikmeli hale getirilmesidir. Operatörün konumundan bağımsız olarak, dünyanın herhangi bir yerindeki bir cihaza anlık müdahale edebilme yeteneği, operasyonel verimlilik ve esneklik açısından kritik bir öneme sahiptir. Geleneksel kontrol yöntemleri olan Bluetooth veya kapalı devre ağ sistemleri, doğaları gereği ciddi kısıtlamalara sahiptir. Bu sistemler, kısa menzil limitleri ve belirli bir ağına dışına çıkamama gibi sorunlar nedeniyle küresel ölçekte uzaktan yönetim senaryoları için yetersiz kalmaktadır. Bu durum, coğrafi olarak dağınık varlıkların merkezileştirilmiş bir şekilde kontrol edilmesini neredeyse imkansız hale getirmektedir. "ControlOverWeb" projesi, bu temel soruna yenilikçi bir çözüm olarak tasarlanmıştır. Projenin birincil hedefi, operatörlerin yalnızca standart bir web tarayıcısı kullanarak, dünyanın herhangi bir yerindeki robotları veya IoT cihazlarını gerçek zamanlı olarak kontrol edebilecekleri bir platform oluşturmaktır. Bu hedefe ulaşmak için sistem, her biri belirli bir sorumluluğu üstlenen çok katmanlı (multi-tier) bir mimari üzerine kurulmuştur. Bu katmanlar; C# .NET Core ile geliştirilen merkezi backend, sistem durumunu yöneten PostgreSQL veritabanı, kullanıcı arayüzünü oluşturan Vanilla JavaScript frontend ve fiziksel komutları işleyen Python tabanlı Raspberry Pi donanım katmanından oluşmaktadır. Bu bölümün ardından, sistemin omurgasını oluşturan veritabanı mimarisi ve yazılım katmanlarının detaylı analizi sunulacaktır.

2. Sistem Mimarisi ve Veritabanı Tasarımı (System Architecture and Database Design)

Bu mimarinin merkezinde, sadece bir veri depolama katmanı olmakla kalmayıp, sistemin anlık durumunu (state) yöneten, işlem bütünlüğünü dayatan ve tüm bileşenler için tekil bir doğruluk kaynağı (single source of truth) görevi gören PostgreSQL veritabanı yer alır. Entity Framework Core ORM (Object-Relational Mapping) yapısı ile yönetilen bu veritabanı, sistemin güvenilirliği, ölçeklenebilirliği ve veri bütünlüğünün temelini oluşturur. Sistemin kararlı çalışmasını sağlayan temel veritabanı tabloları ve kritik işlevleri şunlardır:

- **Receivers (Alicılar/Robotlar)** Bu tablo, sisteme kayıtlı her bir robot veya IoT cihazı için bir kimlik kaydı görevi görür. Robotun adı, tipi (Drone, Rover, Kol vb.) ve anlık

Online/Offline durumu gibi temel bilgileri barındırır. Güvenlik açısından kritik olan SessionPassword sütunu, her bir robotun kendine özel oturum anahtarını saklayarak yetkisiz erişimi engeller. En önemli sütunlardan biri olan LastHeartbeat, robotun sisteme en son ne zaman "hayattayım" sinyali gönderdiğini milisaniye hassasiyetinde kaydeder. Sistem, bu zaman damgasını kullanarak 15 saniye boyunca sinyal alınamayan bir robotu otomatik olarak Offline durumuna geçirir.

- **Senders (Göndericiler/Operatörler)** Kontrol paneline bağlanan operatörlerin geçici oturum bilgilerini ve bağlantı durumlarını yönetir. Bu tablo, hangi operatörlerin hangi robotları izlediğini veya kontrol ettiğini anlık olarak takip etmek için kullanılır.
- **Commands (Komut Kuyruğu)** Sistemin en dinamik ve merkezi işlem tablosudur. Operatör tarafından gönderilen her komut ("Sağa Dön", "Hızlan", "Vektör: 0.5, 1.0" vb.) bu tabloya yeni bir kayıt olarak eklenir. Tablodaki en kritik alan, IsExecuted adındaki boolean (doğru/yanlış) bayrağıdır. Bir komut robota başarıyla iletildiğinde, bu bayrak true olarak güncellenir. Bu mekanizma, ağ gecikmeleri veya tekrarlanan istekler nedeniyle aynı komutun birden fazla kez çalıştırılmasını önleyerek işlem bütünlüğü garanti altına alır.
- **ConnectionHistory (Bağlantı Geçmişi)** Bu tablo, bir güvenlik ve denetim (audit) aracı olarak tasarlanmıştır. Hangi operatörün, hangi robotu, hangi tarih aralığında kontrol ettiğine dair kalıcı kayıtlar tutar. Olası bir arıza, kaza veya yetkisiz kullanım durumunda, "Belirtilen zamanda bu robotu kim yönetiyordu?" sorusuna kesin bir yanıt sunarak sorumluluk takibini mümkün kılar.
- **Logs (Sistem Günlükleri)** Sistemin operasyonel sağlığını izlemek ve olası hataları ayıklamak (debug) için kullanılır. "Robot şifre hatası yaptı" veya "Yeni bağlantı kuruldu" gibi önemli sistemsel olaylar, analiz edilmek üzere bu tabloya kaydedilir.
- **AdminUsers & SiteStats** AdminUsers tablosu, sistemin yönetim paneline erişim yetkisi olan idari kullanıcıları tanımlar. SiteStats tablosu ise toplam ziyaretçi sayısı ve anlık çevrimiçi robot sayısı gibi kümülatif verileri tutarak sistemin genel performansı ve kullanımı hakkında analitik bilgiler sağlar. Bu sağlam veri yapısı, üzerine inşa edilen dinamik yazılım mantığının kararlı ve güvenli bir şekilde çalışması için gerekli zemini hazırlar.

3. Temel Yazılım Mekanizmaları ve Veri Akışı (Core Software Mechanisms and Data Flow)

Sistemin beyni, ASP.NET Core Web API üzerinde çalışan ve tüm veri akışını yöneten yazılım mantığıdır. Bu katman, robotlar ve operatörler arasındaki iletişimin kararlı, gerçek zamanlı ve çakışmasız bir şekilde gerçekleşmesini sağlayan temel algoritmaları ve mimari desenleri barındırır.

Command Polling (Komut Sorgulama) Mimarisi

Robotların genellikle NAT (Network Address Translation) veya güvenlik duvarı (Firewall) gibi ağ katmanlarının arkasında bulunması, sunucunun robota doğrudan bir komut göndermesini teknik olarak zorlaştırır. Bu engeli aşmak için sistem, "Polling" (Sorgulama) adı verilen bir mimari desen kullanır. Doğrudan bağlantı kurmayı gerektiren WebSockets gibi teknolojilerin aksine, Polling mimarisi, iletişimin sorumluluğunu tamamen istemciye (robota) vererek, ağ yapılandırmasından bağımsız, evrensel ve sağlam bir iletişim kanalı garanti eder. İşlem akışı üç basit adımdan oluşur:

1. **Gönderici (Web):** Operatör, kontrol arayüzünde bir eylem gerçekleştirdiğinde (örneğin bir tuşa bastığında), tarayıcı API'ye bir POST isteği gönderir. Bu istek, yeni komutu Commands tablosuna IsExecuted=false olarak kaydeder.
2. **Alıcı (Robot):** Raspberry Pi üzerindeki Python betiği, her 100 milisaniyede bir API'nin /pool adresine bir GET isteği göndererek periyodik olarak "Benim için yeni bir komut var mı?" diye sorar.
3. **İşlem (API):** API, bu isteği aldığı anda veritabanını sorgular. IsExecuted=false durumunda bekleyen bir komut bulursa, bu komutu yanıt olarak robota gönderir ve aynı anda veritabanındaki komutun bayrağını true olarak güncelleyerek işlemi tamamlar. Bu sayede komutun yalnızca bir kez işlenmesi garanti edilir.

"Spy Mode" ile Yarış Durumu (Race Condition) Önleme

Sistemin en özgün mekanizmalarından biri, potansiyel bir kontrol çakışmasını (Race Condition) önlemek için tasarlanan "Spy Mode" mantığıdır. Sorun, bir robotun sisteme kendisini kaydetmesi ile bir operatörün aynı robotun kontrol panelini görüntülemesi arasındaki farkı ayırt etmektir. Eğer bu ayırım yapılmazsa, hem robotun kendisi hem de operatörün tarayıcısı aynı komutları tüketmeye çalışarak istenmeyen davranışlara yol açabilir. Sistem bu sorunu şu şekilde çözer: Bir kullanıcı bir robotun kontrol paneline erişim sağladığında, backend API'si o robotun LastHeartbeat zaman damgasını kontrol eder. Eğer API, robotun en son sinyali (örneğin, "2 saniye önce") gönderdiğini tespit ederse, bu yeni bağlantının robotun kendisi değil, paneli izlemek isteyen bir gözlemci (operatör) olduğu sonucuna varır. Bu durumda API, tarayıcıya özel bir IsSpyMode=true bayrağı gönderir. Bu ayırım, komutları işleme (Consume) yetkisini yalnızca robotun Python betiğine atarken, tarayıcıyı güvenli bir salt-okunur izleme (Watch) moduna zorlayarak kontrol bütünlüğünü mutlak bir şekilde sağlar. Yazılım katmanındaki bu mekanizmalar, komutların dijital dünyadan fiziksel dünyaya aktarılması için donanım katmanına güvenilir bir zemin sunar.

4. Donanım ve Fiziksel Katman Entegrasyonu (Hardware and Physical Layer Integration)

Bu bölüm, ControlOverWeb sisteminin dijital kontrol mantığını fiziksel eylemlere dönüştürdüğü donanım katmanını ele almaktadır. Projenin soyut komutları somut hareketlere çeviren beyni, Raspberry Pi 5 üzerinde çalışan Python betiğidir. Bu betik, bulut altyapısı ile robotun motorları arasındaki kritik arayüzü oluşturur. Python betiğinin temel sorumlulukları şunlardır:

- **Oto-Login:** Betik başlatıldığında, API'ye bağlanarak kendini sisteme tanıtır ve operasyonlar için benzersiz bir Robot ID alır. Bu, robotun komut kuyruğunda kendine ait emirleri ayırt etmesini sağlar.
- **Sinyal İşleme (Signal Processing):** Sunucudan gelen JSON formatındaki komut verilerini (örneğin code: "V:0.5,-1") ayrıştırarak (parsing) anlamlı talimatlara dönüştürür.
- **Vektörel Kontrol (Vector Control):** Sistem, ikili (binary) aç/kapa komutlarının ötesinde, joystick veya dokunmatik arayüzlerden gelen (x, y) gibi orantısal vektör verilerini işleyerek, robot hareketlerinde yüksek hassasiyet ve akıcılık sağlar. Dijital verinin analog harekete dönüşüm süreci şu şekilde işler: Bir operatör joystick'i %50 oranında sağa ittiğinde, bu eylem sunucuya 0.5 gibi sayısal bir değer olarak iletilir. Sunucudan bu veriyi alan Python betiği, bu değeri yorumlar ve karşılık gelen görevi yerine getirmek için Raspberry Pi'nin GPIO (General-Purpose Input/Output) pinlerini kullanır. Örneğin, bu değeri GPIO 18 pinine bağlı bir servo motora belirli bir görev

döngüsüne sahip PWM (Pulse Width Modulation) sinyali olarak gönderir. Bu süreç, soyut bir komut vektörünün, Darbe Genişlik Modülasyonu (PWM) aracılığıyla fiziksel bir aktüatör üzerinde orantısal bir harekete dönüştürülmesini sağlayarak komut-eylem döngüsünü tamamlar.

5. Sonuç (Conclusion)

ControlOverWeb projesi, modern robotik sistemlerin coğrafi sınırlamalardan bağımsız, düşük gecikmeli ve evrensel olarak erişilebilir bir şekilde kontrol edilmesi problemine yönelik başarılı ve kapsamlı bir çözüm sunmaktadır. Geleneksel kontrol paradigmalarının menzil ve kurulum zorluklarını, bulut tabanlı merkezi bir mimari ile etkin bir şekilde aşmaktadır. Projenin temel başarıları; C# .NET Core, PostgreSQL ve Python/Raspberry Pi teknolojilerini bir araya getiren sağlam çok katmanlı mimarisi, ağ bariyerlerini (NAT/Firewall) aşmak için geliştirilen komut sorgulama (polling) mekanizması ve çoklu kullanıcı erişiminde kontrol çakışmalarını önleyen yenilikçi "Spy Mode" mekanizmasıdır. Bu teknik katkılar, yalnızca bir web tarayıcısı kullanarak dünyanın herhangi bir yerinden gerçek zamanlı ve güvenli robot kontrolünü mümkün kılarak, projenin ana hedefine ulaştığını göstermektedir. Sonuç olarak ControlOverWeb, dağıtık robotik sistemlerin yönetimi için yalnızca teorik bir model sunmakla kalmaz, aynı zamanda pratik, uygulanabilir ve ölçeklenebilir bir mühendislik şablonu ortaya koyar.